RAG-Powered Search Engine

Q&A with Retrieval-Augmented Generation

Final Capstone Project · IITG · AI Application Development

LangChain · Azure OpenAI (gpt-5-chat) · local MiniLM embeddings · FAISS · Streamlit

Author: Prasad Thipparthi · Project Option #2

Project Overview & Problem Statement

THE PROBLEM

LLMs cannot answer questions about private or recent documents (knowledge cut-off), and they hallucinate — confident but false answers — when they don't know.

THE GOAL

Answer natural-language questions over a custom document collection; ground every answer in retrieved passages, cite sources, and abstain when the answer is absent.

THE SOLUTION — RETRIEVAL-AUGMENTED GENERATION

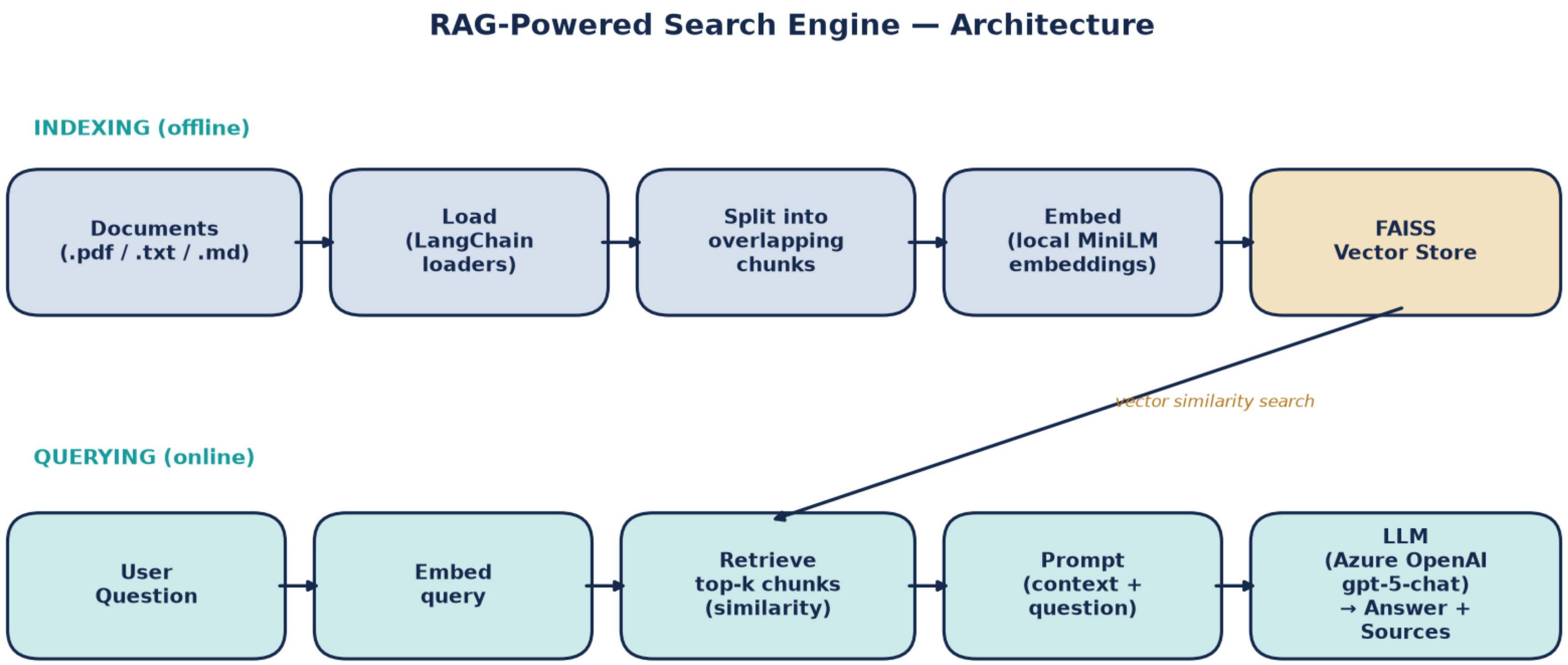
Combine vector similarity search over the documents with an LLM that is instructed to answer only from the retrieved context.

WHY IT MATTERS

Grounding + citations make answers verifiable; explicit abstention is the key anti-hallucination behaviour. Embeddings run locally, so only the chat completion incurs API cost.

ARCHITECTURE

Two-Phase RAG Architecture



Offline INDEXING turns documents into a FAISS vector store; online QUERYING retrieves relevant chunks and generates a grounded, cited answer.

WORKFLOW

How a Question Becomes a Grounded Answer

INDEXING (offline)

- Load — read .pdf / .txt / .md files into LangChain Documents with source metadata.
- Split — RecursiveCharacterTextSplitter → ~1000-char chunks, 150-char overlap.
- Embed — each chunk → dense vector via local all-MiniLM-L6-v2 (offline, free).
- Index — vectors stored in an in-memory FAISS index (persistable to disk).

QUERYING (online)

- Embed query — the question is embedded with the same model.
- Retrieve — FAISS returns the top-k (default 4) most similar chunks.
- Augment — chunks injected into a system prompt that forbids out-of-context answers.
- Generate — Azure OpenAI gpt-5-chat produces a grounded answer + cited sources.

IMPLEMENTATION

Tech Stack & Configuration

Component	Choice	Role
Orchestration	LangChain (LCEL)	Chains load → retrieve → prompt → generate
Chat LLM	Azure OpenAI gpt-5-chat	Generates the grounded answer
Embeddings	all-MiniLM-L6-v2 (local)	Encodes chunks & queries — offline, no cost
Vector store	FAISS (in-memory)	Fast similarity search over chunk vectors
Loaders / splitter	PyPDF + RecursiveCharacterTextSplitter	Ingest & chunk documents
UI / deployment	Streamlit	Upload docs, ask questions, view citations

KEY SETTINGS

chunk_size = 1000 · chunk_overlap = 150 · top_k = 4. Provider is auto-detected from environment variables (Azure → OpenAI fallback); embeddings backend is switchable (local / azure / openai). gpt-5-chat uses its default temperature.

RESULTS

Testing & Evaluation

RETRIEVAL QUALITY — hit@k

For every evaluation question, the expected source document appeared among the top-k retrieved chunks.

5 / 5

hit@4 retrieval

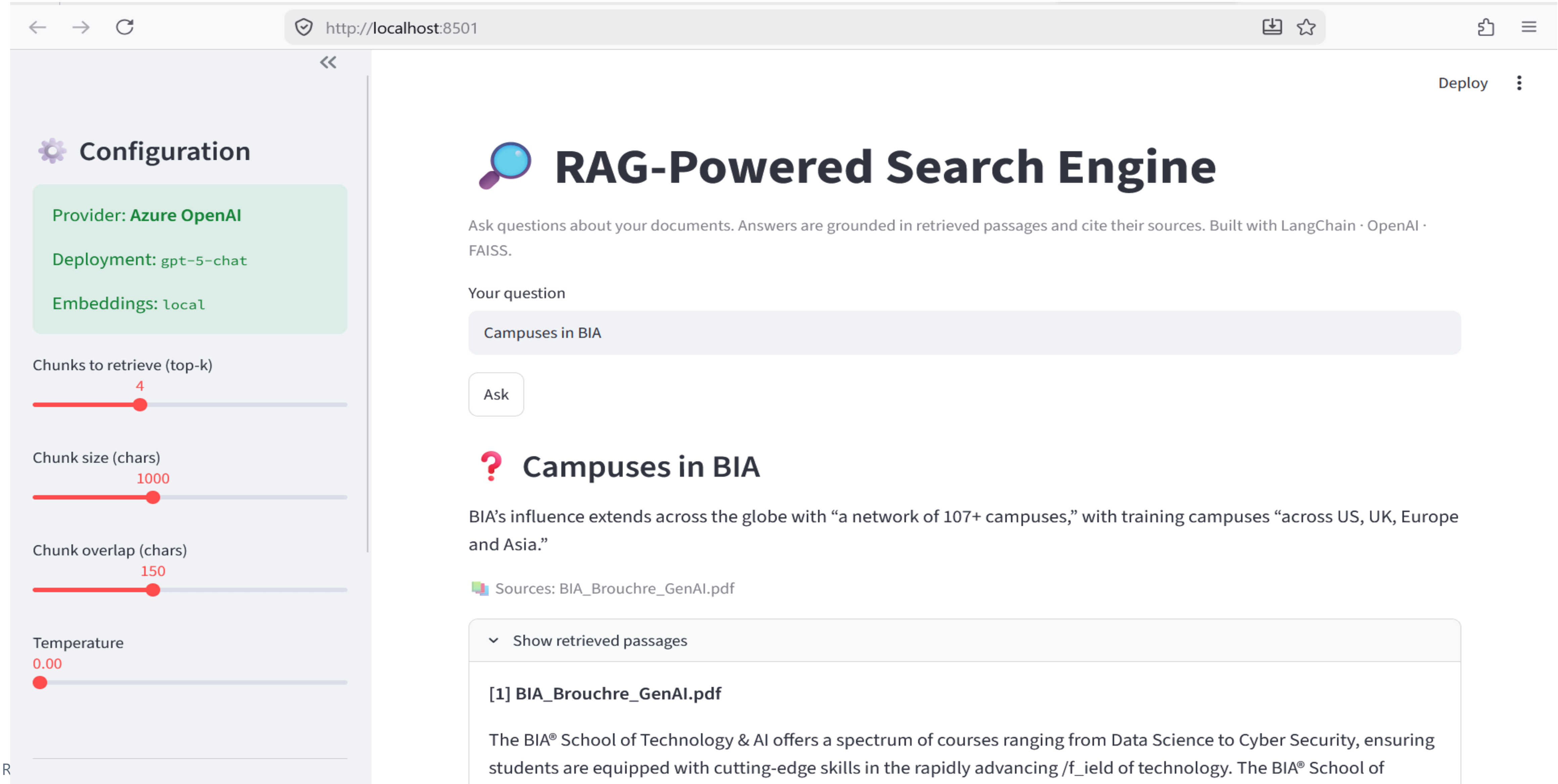
GENERATION QUALITY

In-corpus questions answered correctly with quoted evidence and citations; the out-of-corpus question was correctly refused — "I don't know based on the provided documents."

Evaluation question	Expected source	Retrieved?
What is a token in an LLM?	llm_concepts.md	Yes
Common distance metric for text embeddings?	vector_databases.md	Yes
Good starting chunk size for RAG?	rag_overview.md	Yes
Stock price of Apple on 1 Jan 2026? (out-of-corpus)	— none —	Abstained

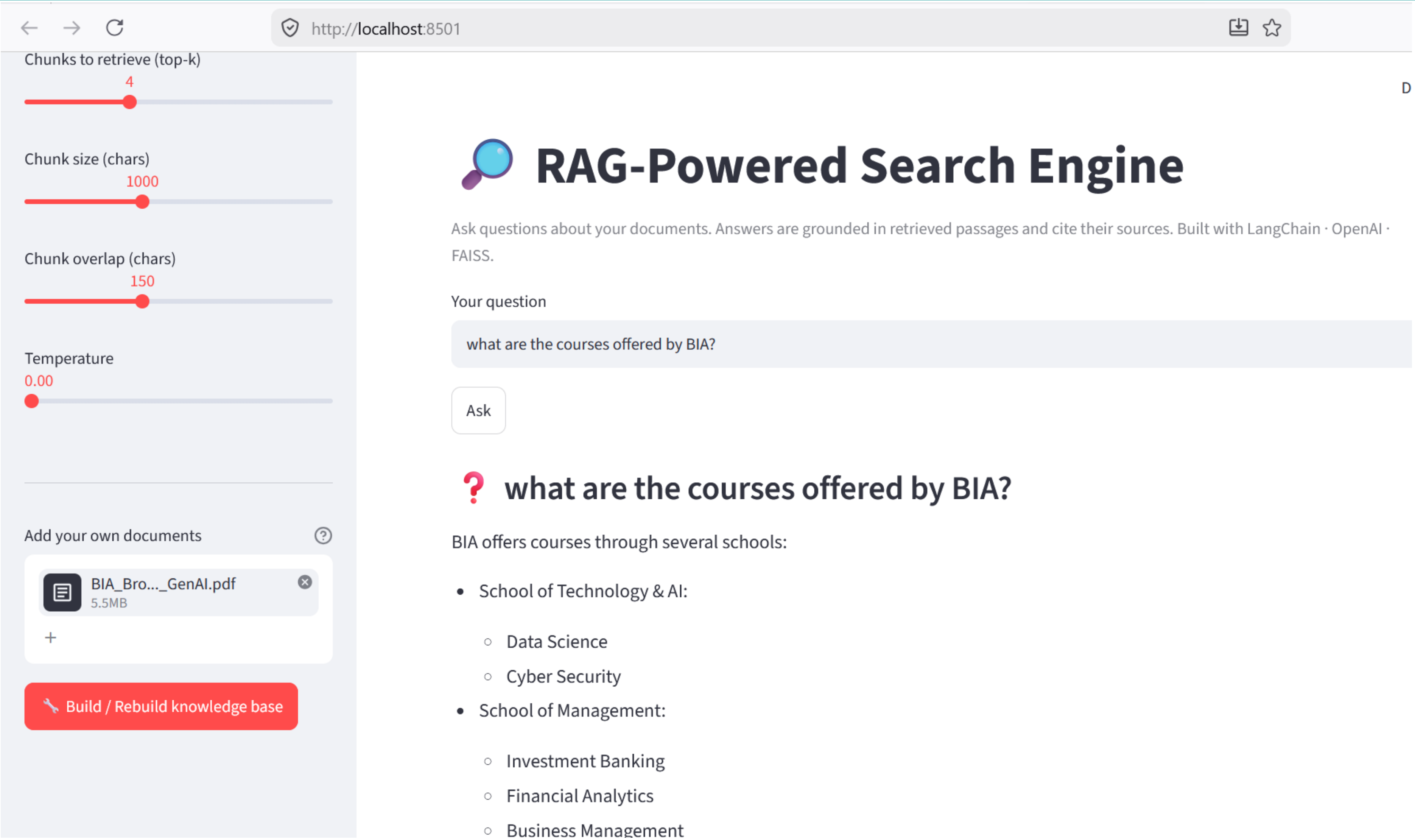
SCREENSHOTS

Demo — Streamlit App



SCREENSHOTS

Demo — Streamlit Results



CHALLENGES

What Was Hard

- Chunking trade-offs — chunks too large dilute relevance; too small lose context. 1000/150 balanced both for this corpus.
- Faithfulness vs. helpfulness — a strict 'answer only from context' prompt can over-abstain; it was tuned to synthesize across retrieved chunks while staying grounded.
- Model constraints — Azure gpt-5-chat rejects temperature=0 (default only); the pipeline omits the parameter on Azure so the call succeeds.
- Cost & latency — running embeddings locally removed embedding API cost; persisting the FAISS index avoids re-embedding on every run.

TAKEAWAY

Most effort went into grounding behaviour and provider quirks, not the happy path.

Future Improvements

- Cross-encoder re-ranking after vector retrieval for higher precision.
- Hybrid search — combine BM25 keyword matching with dense vectors.
- Conversational memory and response streaming for multi-turn Q&A.
- Automated faithfulness scoring with RAGAS.
- Swap FAISS for a managed vector DB (Pinecone / pgvector / Weaviate) at scale.

Thank You

Code, notebook and demo app included in Final_Capstone_Project.zip